Cyclomatic Complexity

Software Engineering Metrics

1 Introduction

Cyclomatic Complexity is a software metric that provides a quantitative measure of the logical complexity of a program.

It was introduced by **Thomas J. McCabe** and is widely used in:

- Software testing
- Code quality assessment
- Risk analysis
- Maintainability evaluation

Cyclomatic complexity defines the number of **independent execution paths** in a program.

2 Why Cyclomatic Complexity is Important

- Provides an upper bound on the number of test cases required for full branch coverage
- Helps identify complex and error-prone modules
- Supports better test planning and maintenance

A higher cyclomatic complexity indicates a more complex and harder-to-maintain program.

3 Independent Paths

An **independent path** is a path through the program that introduces at least one new edge not included in any other independent path.

Cyclomatic complexity gives the **size of the basis set** of independent paths.

4 Notation Clarification (Important for Exams)

To avoid ambiguity:

- P = Number of **connected components** in the control flow graph
- P_d = Number of **predicate (decision) nodes**

5 Methods to Compute Cyclomatic Complexity

There are **three standard methods** to compute cyclomatic complexity.

5.1 Method 1: Using Regions

Cyclomatic complexity is equal to the **number of regions** in the control flow graph.

Each enclosed area in the flow graph (including the outside region) counts as one region.

Method 2: Using Edges and Nodes

For a control flow graph:

$$V(G) = E - N + 2P$$

- E = Total number of edges
- N = Total number of nodes
- P = Number of connected components

Single Method Case

For a single method or function, $P = 1$:

$$V(G) = E - N + 2$$

Strongly Connected Case

If the exit node is connected back to the entry node (strongly connected CFG):

$$V(G) = E - N + P$$

Method 3: Using Predicate Nodes

For structured programs with single entry and exit:

$$V(G) = P_d + 1$$

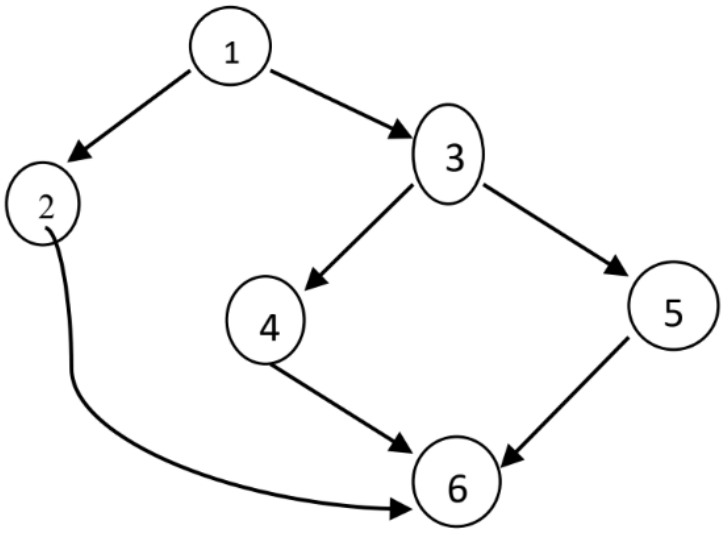
Predicate nodes include `if`, `while`, `for`, etc.

6 Worked Example

6.1 Code Fragment

```
if (a < b)
    F1();
else {
    if (a < c)
        F2();
    else
        F3();
}
```

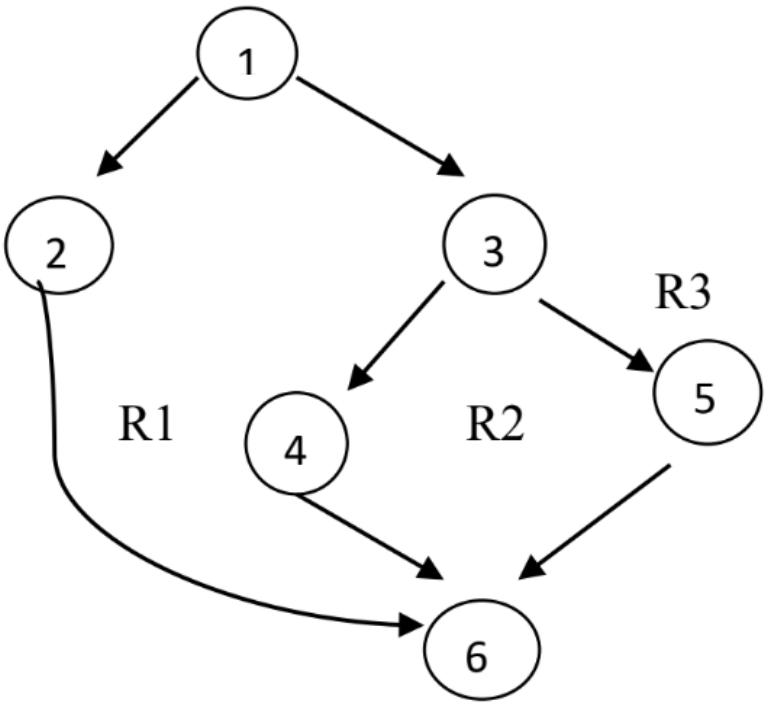
6.2 Step 1: Construct the Flow Graph



The control flow graph contains:

- Conditional branching at line 1
- Nested conditional branching at line 3

6.3 Step 2: Identify Graph Properties



- **Total Nodes (N)** = 6
- **Total Edges (E)** = 7
- **Predicate Nodes (P_d)** = 2 (nodes 1 and 3)
- **Connected components (P)** = 1
- **Total Regions** = 3

6.4 Step 3: Apply Cyclomatic Complexity Formulas

Method 1: Using Regions

$$V(G) = \text{Number of regions} = 3$$

Method 2: Using Edges and Nodes

$$\begin{aligned} V(G) &= E - N + 2P \\ &= 7 - 6 + 2(1) = 3 \end{aligned}$$

Method 3: Using Predicate Nodes

$$V(G) = P_d + 1 = 2 + 1 = 3$$

6.5 Final Result

The cyclomatic complexity of the given code fragment is:

$$\boxed{V(G) = 3}$$

This means:

- There are **3 independent execution paths**
- At least **3 test cases** are required for complete branch coverage

7 Graph-Based Example of Cyclomatic Complexity (Three Methods)

7.1 Code Fragment

```
statement 1;

if (condition 1) {
    statement 2;
} else {
    statement 3;
}

statement 4;

for (condition 2) {
    statement 5;
}

statement 6;
```

7.2 Control Flow Graph

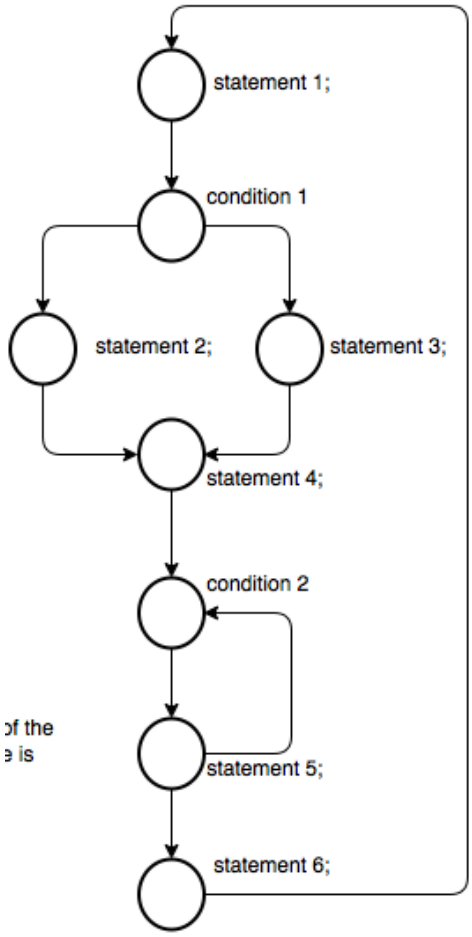


Figure 1: Control Flow Graph with outer back-edge

7.3 Cyclomatic Complexity Calculation

Analysis of Given Graph

Note: The provided diagram includes an extra *outer back-edge* (bottom returning to top). Therefore, the counts below are taken **exactly from the given graph**:

$$N = 8, \quad E = 10, \quad R = 4$$

Step 1: Identify Graph Properties

- Total number of nodes: $N = 8$
- Total number of edges: $E = 10$
- Total number of regions: $R = 4$
- Connected components: $P = 1$
- Predicate nodes (P_d) = 3 (including one diagram-induced cycle)

- **Predicate 1:** `condition 1` (if decision)
- **Predicate 2:** `condition 2` (loop decision)
- **Predicate 3 (as per diagram):** the extra outer loop introduces an additional independent cycle

Step 2: Compute $V(G)$ Using Three Methods

Method 1: Using Regions

$$V(G) = R = 4$$

Method 2: Using Edges and Nodes

$$V(G) = E - N + 2P = 10 - 8 + 2(1) = 4$$

Method 3: Using Predicate Nodes

$$V(G) = P_d + 1 = 3 + 1 = 4$$

7.4 Detailed Formula Clarity (Important)

Context-Dependent Formula

The formula $M = E - N + P$ is CORRECT, but context-dependent.

When the exit point is directly connected back to the entry point, the CFG becomes **strongly connected**, and cyclomatic complexity is:

$$M = E - N + P$$

Parameters:

- E = Number of edges
- N = Number of nodes
- P = Number of connected components

Why does this formula change?

1. Normal Case (Most Programs)

- Entry node has no incoming edge, Exit node has no outgoing edge.
- Graph is **not** strongly connected.

$$M = E - N + 2P \quad \xrightarrow{P=1} \quad M = E - N + 2$$

2. Strongly Connected Case (Your Diagram)

- Exit node connects back to entry (every node is reachable).
- The CFG forms one big cycle.
- **One artificial edge is already present**, so we use $+P$ instead of adding $+2$.

$$M = E - N + P$$

Applying it to YOUR Diagram

From your CFG image ($E = 10, N = 8, P = 1$):

$$M = E - N + P = 10 - 8 + 1 = 3$$

But earlier we got 4, right? Let's explain why.

Why the Mismatch Happens (IMPORTANT)

Your diagram includes two types of cycles:

1. **Real control predicate cycles**: `condition 1` and `condition 2`
2. **Artificial outer back-edge**: `statement 6` $\rightarrow$ `statement 1`

This is often added only for analysis, not an actual decision.

In software testing practice: We do **NOT** reduce complexity just because an artificial back-edge exists. We still count regions, predicate decisions, and independent paths.

Actual Program Logic (Ignoring Artificial Edge)

Predicate nodes $(P_d) = 2$ (`condition 1` and `condition 2`)

$$V(G) = P_d + 1 = 2 + 1 = 3$$

This matches the $E - N + P$ result for strongly connected graphs ($10 - 8 + 1 = 3$), and represents the "clean" logical complexity.

So testers and examiners usually use:

$$M = E - N + 2 = 10 - 8 + 2 = 4$$

Exam-Safe Rule

- **Real program logic back-edge**: Use $M = E - N + P$
- **Artificial back-edge (analysis only)**: Use $M = E - N + 2$

Our diagram's outer back-edge is artificial, therefore: $\boxed{M = 4}$

Final Takeaway

The formula $M = E - N + P$ is theoretically correct. But in exam & testing contexts, unless explicitly stated:

- Treat CFG as single-entry single-exit
- Use $M = E - N + 2$
- Predicate method and region method confirm the final answer

Final Answers (Exam-Safe):

- Connected components (P) = 1
- Regions in the **given CFG** (R) = 4 $\Rightarrow V(G) = 4$
- Real predicate nodes in the **actual code** (P_d) = 2 (if, for) $\Rightarrow V(G) = P_d + 1 = 3$
- **So:** Graph-based complexity = **4**, Code logical complexity = **3**

8 Key Exam Notes

- Cyclomatic complexity measures logical complexity
- It equals the number of independent paths
- Higher value $\Rightarrow$ higher testing effort
- Can be computed using regions, edges–nodes, or predicate nodes